ENCODER INTEGRATION

TECHNICAL DOCUMENTATION

Autonics E50S8-500-3-T-1 × STM32F103 (Blue Pill)

Field	Details
Encoder Model	Autonics E50S8-500-3-T-1
MCU	STM32F103C8T6 (Blue Pill)
Timer Used	TIM2 — Encoder Mode (TI12)
PPR	500 Pulses Per Revolution
Counts Per Rev	2000 (500 PPR × 4 quadrature)
Circumference	10.0 cm per revolution
Sample Window	100 ms
UART Baud Rate	115200
Output Channels	USART1 (external device) + USART2 (serial monitor)
Author / Engineer	_____
Date	_____
Project / Machine	_____

1. Encoder Specifications

The Autonics E50S8-500-3-T-1 is an incremental rotary encoder with 500 pulses per revolution (PPR). Using STM32 hardware quadrature (TI12 mode), the effective resolution is 2000 counts per revolution.

Parameter	Value	Notes
Model	E50S8-500-3-T-1	Autonics incremental encoder
PPR	500	Pulses per revolution
Quadrature Mode	TI12 (×4)	4× resolution
Counts/Rev	2000	500 × 4
Supply Voltage	5V – 24V DC	Check your power supply
Output Type	Push-Pull (T suffix)	Compatible with STM32 GPIO directly at 5V
Shaft Diameter	8 mm	E50S8 series
Max RPM	6000 RPM	Mechanical limit
Output Signals	A, B, Z	Z = index pulse (1/rev), not used here

2. Wiring & Hardware Setup

2.1 Encoder Pin-Out

Encoder Wire Color	Encoder Signal	STM32 Pin	Notes
Brown	+VCC	5V or 12V	Check encoder supply rating
Blue	GND	GND	Common ground with STM32
Black	A	PA0	TIM2_CH1
White	B	PA1	TIM2_CH2
Orange	Z	Not connected	Index pulse — unused in this project

2.2 Critical Wiring Notes

- Both IC1 and IC2 polarities must be set to RISING for correct quadrature direction detection.
- If CW and CCW both read the same direction, swap the A and B signal wires physically on the connector.
- If the encoder supply is 12V, ensure signal outputs are level-shifted to 3.3V before connecting to STM32 GPIO.
- Keep encoder signal cables away from motor drive cables to avoid noise pickup.

2.3 TIM2 Configuration Summary

TIM2 Parameter	Value
Encoder Mode	TIM_ENCODERMODE_TI12
IC1 Polarity	TIM_ICPOLARITY_RISING
IC2 Polarity	TIM_ICPOLARITY_RISING ← CRITICAL (not FALLING)
IC1 Filter	15 (maximum noise filtering)
IC2 Filter	15 (maximum noise filtering)
Prescaler	0
Period (ARR)	65535 (full 16-bit range)
Counter Mode	TIM_COUNTERMODE_UP

3. Software Architecture

3.1 Key Defines

Define	Value	Purpose
COUNTS_PER_REV	2000	500 PPR × 4 quadrature edges
CIRCUMFERENCE_CM	10.0f	cm per full shaft revolution
SAMPLE_MS	100	Sampling window in milliseconds (10 Hz update rate)
MIN_DELTA	1	Noise deadband threshold in counts
EMA_ALPHA	0.5f	EMA filter coefficient — 0=heavy smooth, 1=raw

3.2 Global Variables

Variable	Type	Purpose
g_total_counts	int32_t	Accumulated signed count — tracks absolute position
g_prev_count	int32_t	Previous sample counter value for delta calculation
g_rpm_filtered	float	EMA-filtered RPM output

3.3 Encoder_Process() — Step by Step Logic

This function is called every SAMPLE_MS (100 ms) from the main loop.

Step 1 — Read Hardware Counter

```
int32_t current = (int32_t)__HAL_TIM_GET_COUNTER(&htim2);
```

Reads the raw 16-bit TIM2 counter register.

Step 2 — 16-bit Overflow Correction

```
int32_t delta = current - g_prev_count;
if      (delta >  32767) delta -= 65536;
else if (delta < -32768) delta += 65536;
```

Handles counter wrap-around at 0 and 65535 correctly. Without this, a wrap from 65535→0 would appear as a -65535 jump instead of +1.

Step 3 — Noise Dead-band

```
if (delta > -MIN_DELTA && delta < MIN_DELTA) delta = 0;
```

Rejects tiny movements below the threshold (1 count) caused by electrical noise or mechanical vibration.

Step 4 — Accumulate Position

```
g_prev_count    = current;
g_total_counts += delta;
```

Signed accumulation: CW rotation adds positive counts, CCW rotation adds negative counts, giving a true absolute position tracker.

Step 5 — Raw RPM Calculation

```
float sample_sec = (float)SAMPLE_MS / 1000.0f;
float rpm_raw = (fabsf((float)delta) / COUNTS_PER_REV) / sample_sec * 60.0f;
```

Converts counts in the sample window to revolutions per minute. Uses fabsf() so RPM is always positive regardless of direction.

Step 6 — EMA Filter

```
if (delta == 0)
    g_rpm_filtered = 0.0f;                                  // Hard zero on stop
else
    g_rpm_filtered = EMA_ALPHA * rpm_raw
                   + (1.0f - EMA_ALPHA) * g_rpm_filtered;
```

Exponential Moving Average smooths noisy raw RPM readings. When the shaft stops (delta == 0), RPM is forced immediately to 0.0 — no slow decay.

Step 7 — Distance Calculation

```
float distance_cm = ((float)g_total_counts / COUNTS_PER_REV) * CIRCUMFERENCE_CM;
```

Converts accumulated signed counts to cm. Positive = CW travel, Negative = CCW travel from start position.

Step 8 — UART Output

```
[CW ] RPM:  125.76  |  Distance:    20.430 cm
```

Transmitted to both USART1 (external device) and USART2 (serial monitor) simultaneously.

4. Bugs Fixed During Development

This section documents every bug encountered and resolved — valuable context for future maintainers.

#	Problem	Root Cause	Fix Applied
1	Distance always increased regardless of direction	g_rpm_filtered * (1-alpha) decay never zeroed RPM; direction sign lost	Hard zero: g_rpm_filtered = 0.0f when delta == 0
2	RPM unstable, large random spikes	Single 500ms sample window — any speed variation caused huge jumps	Added EMA filter (alpha=0.5) + reduced SAMPLE_MS to 100ms
3	1–2 second delay in readings	SAMPLE_MS was 500ms + EMA alpha=0.2 added more lag	Changed SAMPLE_MS to 100ms, alpha to 0.5f
4	RPM decayed slowly after shaft stopped (ghost readings)	EMA formula g * (1-alpha) never reached zero	Force g_rpm_filtered = 0.0f when delta == 0
5	CCW rotation showed [CW] and distance never decreased	IC2Polarity set to FALLING — breaks quadrature direction detection in TI12 mode	Changed IC2Polarity to TIM_ICPOLARITY_RISING
6	Both directions read same sign	delta = -delta negation applied when not needed for this wiring	Keep delta = -delta line; fix polarity in hardware config instead
7	EMA_ALPHA had no effect — defined inside function	#define inside function compiles but cannot be tuned from top of file	Moved EMA_ALPHA to top-level #define section with other constants

5. Tuning Reference

5.1 SAMPLE_MS Tradeoff

SAMPLE_MS	Update Rate	RPM Resolution	Suitable For
50 ms	20 Hz	Low at slow speeds	High-speed motors, fast response needed
100 ms	10 Hz	Good balance	General purpose — recommended
200 ms	5 Hz	Better low-speed RPM	Slow machines, precision distance tracking
500 ms	2 Hz	Best low-speed accuracy	Very slow or near-static measurements

5.2 EMA_ALPHA Tuning

EMA_ALPHA	Smoothing	Response Speed	Best Use
0.1	Very heavy	Slow	Very noisy signal, stable speed only
0.2	Heavy	Slow	Noisy environments
0.5	Moderate	Medium	General purpose — recommended
0.7	Light	Fast	Clean signal, need quick response
1.0	None (raw)	Instant	Debug only — shows raw RPM

5.3 Direction Verification Checklist

- Rotate shaft CW → Distance should increase (positive), direction tag shows [CW]
- Rotate shaft CCW → Distance should decrease (negative or back toward 0), tag shows [CCW]
- Stop shaft → RPM drops to 0.00 immediately, distance holds steady, tag shows [---]
- If both rotations show [CW]: IC2 polarity is wrong — set to TIM_ICPOLARITY_RISING
- If directions are swapped (CW reads CCW): swap A and B wires on connector, OR add/remove delta = -delta

6. Complete Encoder_Process() Code

Copy-paste ready final version of the encoder processing function:

```
/* --- Top-level defines (in USER CODE BEGIN PD) --- */
#define COUNTS_PER_REV    2000
#define CIRCUMFERENCE_CM  10.0f
#define SAMPLE_MS         100
#define MIN_DELTA         1
#define EMA_ALPHA         0.5f

/* --- Global variables (in USER CODE BEGIN PV) --- */
int32_t g_total_counts = 0;
int32_t g_prev_count   = 0;
float   g_rpm_filtered = 0.0f;

/* --- Function --- */
void Encoder_Process(void)
{
    char     buf[80];
    uint16_t len;

    int32_t current = (int32_t)__HAL_TIM_GET_COUNTER(&htim2);

    int32_t delta = current - g_prev_count;
    if      (delta >  32767) delta -= 65536;
    else if (delta < -32768) delta += 65536;

    delta = -delta;  /* remove if direction wrong for your wiring */

    if (delta > -MIN_DELTA && delta < MIN_DELTA)
        delta = 0;

    g_prev_count    = current;
    g_total_counts += delta;

    float sample_sec = (float)SAMPLE_MS / 1000.0f;
    float rpm_raw    = (fabsf((float)delta) / (float)COUNTS_PER_REV)
                       / sample_sec * 60.0f;

    if (delta == 0)
        g_rpm_filtered = 0.0f;
    else
        g_rpm_filtered = EMA_ALPHA * rpm_raw
                       + (1.0f - EMA_ALPHA) * g_rpm_filtered;

    float distance_cm = ((float)g_total_counts / (float)COUNTS_PER_REV)
                        * CIRCUMFERENCE_CM;

    const char *dir = (delta > 0) ? "CW " : (delta < 0) ? "CCW" : "---";
```

```
        len = snprintf(buf, sizeof(buf),
                       "[%s] RPM: %7.2f  |  Distance: %9.3f cm\r\n",
                       dir, (double)g_rpm_filtered, (double)distance_cm);

        Send_Both(buf, len);
    }
```

7. Handover Notes

7.1 What the Next Engineer Must Know

- IC2 polarity MUST be RISING — setting it to FALLING silently breaks direction detection. This was the hardest bug to find.
- The delta = -delta line may or may not be needed depending on how A/B are wired to the board. If both directions read the same, try removing it.
- EMA_ALPHA must be defined at the top level (USER CODE BEGIN PD), not inside the function, or changes have no effect.
- SAMPLE_MS drives both the update rate AND the RPM calculation denominator — changing it automatically adjusts RPM math (no other changes needed).
- g_total_counts accumulates indefinitely — it does not reset unless you call the init sequence again. Reset it deliberately if you need relative position from a new zero point.

7.2 Quick Fault Diagnosis

Symptom	Most Likely Cause	Action
Distance always increases	Direction detection broken	Check IC2 polarity — must be RISING
Both dirs show [CW]	IC2 polarity = FALLING	Change to TIM_ICPOLARITY_RISING in MX_TIM2_Init
RPM decays slowly after stop	EMA decay not zeroed on stop	Ensure delta==0 branch sets g_rpm_filtered = 0.0f (not *= factor)
RPM very unstable / spiky	SAMPLE_MS too large or alpha too low	Reduce SAMPLE_MS to 100, increase EMA_ALPHA to 0.5
No counts at all	Wiring or power issue	Check VCC/GND, probe A and B signals with oscilloscope
Counts but wrong direction	A and B wires swapped	Swap A and B wires on connector
Values delayed 1-2 seconds	SAMPLE_MS = 500, alpha = 0.2	Change SAMPLE_MS=100, EMA_ALPHA=0.5f

7.3 Files & Locations

- Main source: Core/Src/main.c
- Peripheral config: MX_TIM2_Init(), MX_USART1_UART_Init(), MX_USART2_UART_Init()
- Encoder logic: Encoder_Process() — USER CODE BEGIN 4 section
- Global state: USER CODE BEGIN PV section
- Defines: USER CODE BEGIN PD section

End of Document

Keep this document updated whenever the encoder setup changes.